Skill Gap & Job Fit Analyzer — Full Project Reference

Your complete build log — what it is, how it works, and every file explained.

What This Project Is

A Python web app that takes a Job Description and a Resume (PDF), runs AI analysis on both, and gives you:

- A **Fit Score** (semantic similarity via SBERT embeddings)
- A **Skill Gap breakdown** (keyword scan against a master IT skills list)
- A **Career Coach chatbot** powered by Google Gemini for follow-up questions

Built as a final year project. Stack: Python · Streamlit · Sentence Transformers · pdfplumber · Google Gemini API.

Project File Structure

```
project/
├── app.py              # Streamlit UI — all visual components
├── engine.py           # Backend brain — model, scoring, LLM calls
├── requirements.txt    # All pip dependencies
├── .env                # Your Gemini API key (never share this)
├── .gitignore          # Excludes .env, __pycache__, venv
└── README.md           # Setup guide for anyone running it fresh
```

How It Was Built — Step by Step

Step 1 — Model Load + PDF Parser

Loaded `all-MiniLM-L6-v2` from `sentence-transformers` and wrote `get_resume_text(file_path)` using `pdfplumber` to extract all text from a PDF page by page. Tested by encoding a sample string and printing the embedding shape — expected `(384,)`.

Step 2 — Fit Score (Cosine Similarity)

Wrote `calculate_fit_score(resume_text, job_description)`. Both texts are encoded into 384-dimension vectors using the SBERT model. `util.cos_sim()` from `sentence_transformers` calculates the cosine angle between them. Result multiplied by 100 and rounded to give a percentage.

Key detail: `util.cos_sim` returns a 2D tensor `[[score]]` even for single pairs, so you index `[0][0]` to get the scalar.

Step 3 — Skill Gap Analysis

Wrote `identify_skill_gaps(resume_text, job_description)` with a master `it_skills` list split into `data_ai_skills` and `software_dev_skills`.

Logic:

1. Scan the JD first to find which skills are *required*
2. Scan the resume to find which of those it *has*
3. Return `matched_skills` and `missing_skills`

Used `re.escape` + `\b` word boundaries so multi-word skills like `delta lake`, `node.js`, `rest api` match correctly without false positives.

Step 4 — Chatbot Loop (if/elif version)

Combined all functions into `analyze_job_fit()` and built a `while True` terminal chatbot with `if/elif` intent matching. Handled intents: score explanation, missing skills, suggestions, category, certifications. This was the pre-LLM version.

Step 5 — LLM Upgrade (Google Gemini)

Replaced all `elif` blocks with a single `ask_llm()` call. Analysis context (score, skills, resume, JD) is injected into every prompt as a fixed block. `chat_history` is maintained and passed in so Gemini remembers earlier turns. Model used: `gemini-1.5-flash` (free tier).

Step 6 — Modularised into engine.py

All logic moved to `engine.py` as clean exported functions. Private internals prefixed with `_`. Single entry point `run_analysis()` returns one dict with everything `app.py` needs. `get_ai_guidance(question, context_data)` handles all LLM calls.

Step 7 — Streamlit UI (app.py)

Built the full web UI. Sidebar for PDF upload. Main area for JD input. Analyze button (disabled until both inputs filled). Results section with score card, category badge, score bar, and skill badges. Chat interface using `st.chat_input` and `st.session_state`.

---

engine.py — Full Explanation

Startup

```
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
_embed_model = SentenceTransformer("all-MiniLM-L6-v2")
_llm = genai.GenerativeModel("gemini-1.5-flash")
```

Runs once on import. Loads `.env`, authenticates Gemini, loads SBERT into memory. Underscore prefix = internal, don't call from outside.

`get_resume_text(file_path)`

Opens PDF with pdfplumber, loops every page, concatenates text. Works with both a file path string and a Streamlit `UploadedFile` object — pdfplumber handles both natively.

`calculate_fit_score(resume_text, job_description)`

Core ML function. Encodes both strings into 384-dim vectors, measures cosine similarity. Returns percentage. This is **semantic** — understands meaning, not just keywords. So "data pipeline" ≈ "ETL workflow" even though the words differ.

`identify_skill_gaps(resume_text, job_description)`

Keyword scan using regex. Scans JD first (not the whole skill list) to avoid false gaps. Returns two sorted lists: `matched` and `missing`.

`get_fit_category(score)`

Score	Category
⩾ 80%	Strong Fit
50–79%	Potential Fit
< 50%	Low Fit

`get_ai_guidance(question, context_data)`

Builds a structured prompt with score, skills, resume, and JD baked in. Accepts `chat_history` for multi-turn memory. Sends to Gemini, returns reply string. System prompt instructs it to act as a career mentor focused on Data & AI / Software Dev only.

`run_analysis(resume_source, job_desc)`

The one function `app.py` calls. Chains all four functions and returns a single dict:

```
{
  "score":       73.5,
  "category":    "Potential Fit",
  "matched":     ["python", "sql", "git"],
  "missing":     ["pyspark", "airflow", "docker"],
  "resume_text": "...",
  "job_desc":    "..."
}
```

---

app.py — Full Explanation

Session State

```
st.session_state.results       # dict from run_analysis()
st.session_state.chat_history  # list passed to Gemini for memory
st.session_state.messages      # list for rendering chat bubbles
st.session_state.analyzed      # bool — controls results visibility
```

Streamlit reruns the entire script on every user interaction. Session state is how data persists across those reruns.

Sidebar

`st.file_uploader` for PDF. Status message shows filename on upload. Analyze button is `disabled=not (uploaded_pdf and job_desc.strip())` — stays greyed out until both inputs are filled.

On Analyze Click

Calls `engine.run_analysis()`, stores result in session state, resets chat history so a re-analysis clears old conversation.

Results Rendering

Score card and category badge use raw HTML via `st.markdown(..., unsafe_allow_html=True)` because Streamlit's native components don't allow enough color control. Skill badges (green for matched, red for missing) use the same approach.

Chat Interface

`st.chat_input` captures questions. On each message:

1. User bubble renders immediately
2. `engine.get_ai_guidance()` called with results + history
3. AI reply renders
4. Both appended to `messages` (display) and `chat_history` (LLM memory)

Two separate lists because display format and Gemini's expected format differ slightly.

The Skill Lists

Two lists combined into one master `it_skills`:

Data & AI: python, sql, pandas, numpy, scikit-learn, tensorflow, pytorch, pyspark, spark, kafka, airflow, dbt, azure, aws, gcp, databricks, snowflake, bigquery, power bi, tableau, delta lake, machine learning, deep learning, nlp, llm, mlops, etl, medallion architecture, and more.

Software Dev: java, javascript, typescript, react, node.js, fastapi, flask, django, docker, kubernetes, terraform, git, ci/cd, rest api, graphql, microservices, linux, bash, redis, and more.

Data Flow — End to End

```
User uploads PDF + pastes JD
          ↓
   engine.run_analysis()
          ↓
   get_resume_text()        → extracts raw text from PDF
   calculate_fit_score()    → SBERT embeddings → cosine sim → %
   identify_skill_gaps()    → regex scan → matched + missing lists
   get_fit_category()       → Strong / Potential / Low
          ↓
   st.session_state.results  (stored for chat to use)
          ↓
   Score card + badges render in UI
          ↓
   User types question in st.chat_input
          ↓
   engine.get_ai_guidance(question, {results + chat_history})
          ↓
   Gemini replies → renders in chat bubble
   Appended to chat_history for next turn's memory
```

Tech Stack Summary

Component	Library / Service
Web UI	Streamlit
Semantic similarity	sentence-transformers (`all-MiniLM-L6-v2`)
PDF parsing	pdfplumber
AI chatbot	Google Gemini (`gemini-1.5-flash`)
Env variables	python-dotenv

Requirements.txt

```
streamlit
sentence-transformers
pdfplumber
python-dotenv
google-generativeai
```

Install: `pip install -r requirements.txt`

Pin versions before final submission: `pip freeze > requirements.txt`

Running the App

```
# Activate venv first
venv\Scripts\activate          # Windows
source venv/bin/activate       # Mac/Linux

# Run
streamlit run app.py
```

Opens at `http://localhost:8501`

Key Design Decisions Worth Remembering

Why scan JD first, not the full skill list — avoids false gaps. No point flagging Rust as missing if the job never asked for it.

Why two history lists in app.py — `messages` is for Streamlit's chat UI format, `chat_history` is for Gemini's role/content format. They're slightly different structures.

Why engine.py is stateless — all state lives in `st.session_state` in app.py. Engine just takes inputs and returns outputs. Clean separation.

Why `run_analysis()` as a single entry point — app.py makes one call and gets everything back as a dict. Easy to cache with `@st.cache_data` later if needed.

Semantic vs keyword — the fit score uses meaning (SBERT), the skill gap uses keywords (regex). Both approaches together give a more complete picture than either alone.